

An Exploratory Take on Feed-Forward Neural Networks and Their Applications: A Case Study in Regression and Classification Analysis

John-Magnus Johnsen*

Simen L. Wærstad

University of Oslo

(Dated: November 10, 2025)

In data analysis, ANNs are used to create predictive models on complex data sets. MLPs are a class of fully connected feed-forward neural networks with non-linear activation functions. MLPs perform better than traditional regression models in many cases, but can be significantly more computationally expensive. Even if MLPs are necessary, the cost can vary greatly depending on the number and size of the hidden layers. Hence, it is of interest to determine in what cases traditional regression methods are sufficient and how to manage the cost and complexity trade-off of MLPs when they are necessary. Here we show that OLS was outperformed by MLPs in a non-linear case. In addition, we showed that MLPs of more nodes and layers did not necessarily outperform simpler models. By implementing a flexible FFNN with back propagation from scratch we compared the performance of MLPs of various sizes with OLS. This comparison was done by modeling the Runge function. In addition we found that the number of layers and nodes had different impact for different activation functions. In particular, sigmoid and ReLU seemed to suffer from vanishing gradients. Our results show that ANNs are not the solution for all problems and that there are cases where traditional regression methods are preferable. We find that our best FFNN model get an MSE on test data of 0.00861, while OLS get 0.00946. When ANNs are preferable our data shows that the choice of architecture and hyperparameters are important for creating an optimal model and that larger networks are not always better.

I. INTRODUCTION

Inspired by the neurons in our brains, artificial neural networks (ANNs) have proved to be powerful tools across every field of science. They have been used to predict the folding of proteins in biochemistry [1], finding singularities in solutions to Navier-Stokes equation in fluid-mechanics [2], and aiding in drug discovery in the pharmaceutical sciences [3].

While ANNs are a powerful and popular tool, it can be a very expensive and complicated process to create an appropriate model. In contrast, traditional regression methods are less computationally intensive and in many cases relatively simple. Thus, it is of interest to determine when traditional regression methods are sufficient and when ANNs are necessary. Furthermore, when ANNs are necessary it is desirable to use the computationally cheapest and simplest models possible.

To investigate this we implemented our own feed forward neural network (FFNN) with backpropagation and compared it with the traditional regression methods. To verify that our implementation is valid we compared it to the standard machine learning library scikit-learn.

The comparison between our FFNN implementation and the traditional regression methods was done by modeling real functions. In particular, we considered the Runge function. Additionally, we tested our FFNN implementation on a classification problem using the MNIST data set. For the FFNN we considered various numbers

of nodes and hidden layers as well as different activation functions.

This article is structured as follows: In section II we consider the theory of FFNNs and specifics of our implementation; In section III we present and analyze the results from comparing OLS with FFNN models of various sizes. We also look at the behavior of different activation functions on a classification model; Finally, in section IV Our conclusions and suggestions for future work are given.

II. METHODS

A. Theory

The following theory section is largely based on notes by M. Hjorth-Jensen [4].

1. Feed Forward Neural Network

A feed forward neural network is a set of connected nodes, where the nodes are connections are one-directional. The nodes are organized into layers. The nodes in each layer are not connected, but is connected with every node in the previous and next layer. The first layer is usually called the input layer and the last layer is called the output layer. The layers between the input layer and the output layers are called the hidden layers. A illustration of the structure of a simple feed forward network can be seen in figure 1.

Each node does a simple set of operations. It takes in

*https://github.com/johnmagj/FYS-STK4155_Project_2

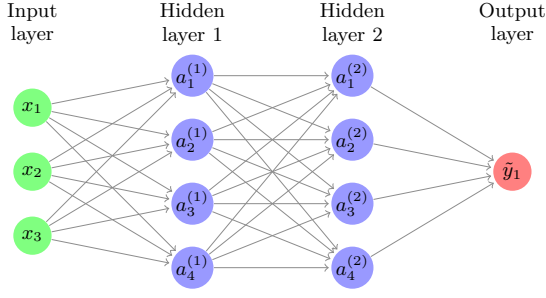


Figure 1: A simple FFNN architecture. Three input features are simultaneously fed into the network and propagated forward layer by layer, culminating in a single output value after activation in the output node.

the outputs $a_j^{(l-1)}$ of the nodes from the previous layer ($l - 1$) and then takes the weighted sum of them before applying an activation function σ . Or stated mathematically, the weighted sum at the j th node of the l th layer is given by

$$z_j^{(l)} = \sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)}$$

where w_{ij} is the weight of the output of the i th node of the previous layer at the j th node of the l th layer, and $b_j^{(l)}$ is the bias of the j th node of the l th layer. Thus the output of the layer is given by

$$a_i^{(l)} = \sigma^{(l)}(z_i)$$

where $\sigma^{(l)}$ is the activation function of the l th layer. Thus the previous layer feeds it input forward to the next layer after having done some computations.

The nodes in the input layer takes what ever data we provide the network. Therefore the number of input nodes need to match the dimensionality of the input data. Similarly, the output nodes is where we read off the networks output, and hence, the size of the output layer needs to match the size of our desired output data.

Note that if the activation functions σ are linear, then the network reduces to taking linear combinations of the input values. Hence, the σ values must be non-linear functions.

The output of the feed forward network depends heavily on the weights and biases, and therefore we need a way to select them such that our network can do anything of use. This leads us to the next section on backpropagation.

2. Backpropagation

An integral part of a feed forward neural network model is how finding the right weight and biases and produce a useful model. One of of doing this is backpropagation.

Similarly to traditional regression models we can create a cost function that gives the cost (that is the performance of) as a function of the weights and biases. Thus we can minimize the cost by taking the derivative of cost function with respect to the weights and approximate the optimal choice of weights and biases by gradient descent (or some variation thereof).

Due to the structure and size of neural networks computing the gradients of the cost function can be complicated. Backpropagation, however, allows for iterative application of the chain rule, greatly simplifying the process.

Denoting the Cost function by C we can calculate the error in the output layer $\delta^{(L)}$ by

$$\delta_j^{(L)} = \frac{\partial C}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \sigma'(z_j^{(L)}) \frac{\partial C}{\partial a_j^{(L)}}.$$

Then we can calculate the derivative with respect to the weights by

$$\frac{\partial C}{\partial w_{ij}^{(L)}} = \frac{\partial C}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial z_j^{(L)}}{\partial w_{ij}^{(L)}} = \delta_j^{(L)} a_i^{(L)},$$

and the derivative with respect to the biases by

$$\frac{\partial C}{\partial b_i^{(L)}} = \frac{\partial C}{\partial b_j^{(L)}} \frac{\partial b_j^{(L)}}{\partial z_j^{(L)}} = \delta_j^{(L)},$$

since

$$\frac{\partial b_j^{(L)}}{\partial z_j^{(L)}} = 1.$$

For subsequent layers we have that

$$\delta_j^{(l)} = \sum_k \frac{\partial C}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)} \sigma'(z_j^{(l)})$$

and thus

$$\frac{\partial C}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} a_i^{(l-1)},$$

and

$$\frac{\partial C}{\partial b_j^{(l)}} = \delta_j^{(l)}.$$

These equations require the activation and node values (that is, the $a_i^{(l)}$ and $z_i^{(l)}$). These are, however, already computed during the forward feed pass. Therefore, simply storing these intermediate values during a forward feed pass makes it possible to compute the the gradients using backpropagation at little extra cost.

3. Cost Functions

Like with classical regression methods the choice of cost function is important. In the case of regression we can use the familiar mean squared error function (MSE):

$$C(\theta) = \frac{1}{N} \sum_i^N (\tilde{y}_i - y_i)^2 \quad (1)$$

When using the MSE function for regression one typically uses a linear function in the output layer.

In the case of classification problems we require another cost function. In classification problems we typically use the negative log-likelihood:

$$C(\theta) = -\log P(D | \theta)$$

where $P(D | \theta)$ is the probability of observing the data D given weights θ . When used in combination with the softmax function as the activation function in the output-layer it can be shown that the error in the output layer reduces to

$$\delta^{(L)} = y - \hat{y}.$$

As we will only use cross-entropy and softmax in conjunction we do not need to calculate their derivatives separately, which can be quite a burdensome calculation.

4. Activation Functions

As briefly mentioned in the subsection on FFNNs (subsection IIA 1), FFNNs are only useful if the functions are non-linear. This section will briefly discuss a few commonly used activation functions.

The sigmoid function is given by

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}.$$

It takes values between 0 and 1. As $x \rightarrow \infty$ we have that $\text{sigmoid}(x) \rightarrow 1$, and as $x \rightarrow -\infty$ we have that $\text{sigmoid}(x) \rightarrow 0$. The derivative of the sigmoid function is given by

$$\text{sigmoid}' = \text{sigmoid}(x)(1 - \text{sigmoid}(x)).$$

The ReLU function is given by

$$\text{ReLU} = \begin{cases} x, & x > 0 \\ 0, & x \leq 0. \end{cases}$$

It's derivative is similarly, given by

$$\text{ReLU}' = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0. \end{cases}$$

If $x > 0$, ReLU is simply the identity, if $x \leq 0$ it is the constant function 0. This has a unfortunate consequence.

If the weights make the value at a node become negative the activation becomes 0 and so does its derivative. This can cause the node to "die", that is it might become stuck at a constant 0 output as the weights are unlikely to update when the gradient is essentially constantly 0.

The leaky ReLU function is a small modification of ReLU that aims to address the problem of vanishing gradients. It achieves this by changing the output value when $x \leq 0$. The leaky ReLU function is given by

$$\text{ReLU} = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0, \end{cases}$$

and it's derivative is

$$\text{ReLU}' = \begin{cases} 1, & x > 0 \\ \alpha, & x \leq 0. \end{cases}$$

Here, α is some small value, typically 0.01, which is what we will use in this paper.

Finally, we have the softmax function:

$$\text{softmax}(x) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}},$$

where $x = [x_1, \dots, x_N]$. For our purposes we do not need its derivative.

B. Dataset and pre-processing

For the regression analysis we generated a dataset by adding noise to the Runge function, this by sampling normally distributed values with mean zero, and standard deviation 0.1. The dataset were then split into a train and test set, 80-20. A visualization of both splits, and the underlying function can be seen in figure 2.

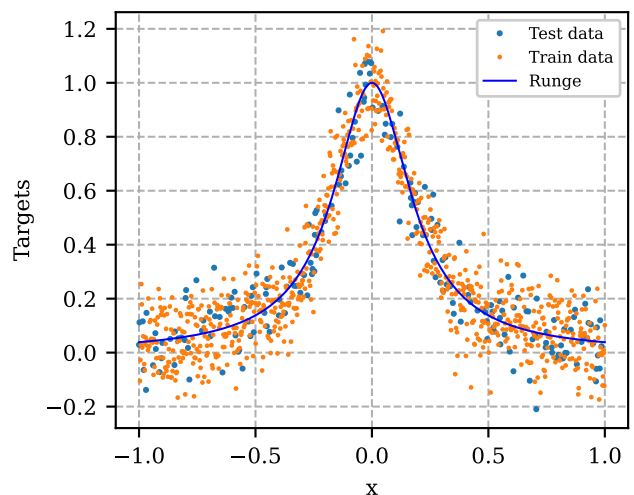


Figure 2: The dataset and the Runge function.

The input features were scaled by standardization. While not so common in standard linear regression, the

target values were also scaled. This is done to minimize a large spread in the values used during training and thus hopefully aid with a quicker convergence of the model parameters.

C. Implementation

In order to carry out the comparisons we implemented our own FFNN with backpropagation. Our class-based implementation is flexible and allows for a user-defined neural network architecture. The number of layers and nodes per layers is can be freely chosen, and it is possible to pick different activation functions for hidden layer as well as the output layer. In addition, the cost function can be freely swapped out. The neural network class also has a classification mode that always uses the cross-entropy as the cost function and softmax as the output function in order to simplify the gradient calculations.

In order to allow verification of the gradient calculations the FFNN was implemented using autograd’s wrapper of NumPy [5] to enable comparison with the gradients computed by autograd [6]. Comparison showed that the difference was insignificantly small and outside floating point precision.

In order to verify that our implementation was working it was compared to the multi-layer perceptrons implemented by the python library scikit-learn[7].

First, we compared our FFNN with scikit-learn’s `MLPRegressor` by modelling the Runge function. The feature-data was generated by sampling 1000 points from $N(0,1)$. The corresponding targets were generated by applying Runge’s function to the targets and adding noise distributed as $N(0,0.1)$. The data was then randomly split into training and test sets with a test proportion of 0.2. All the models consisted of an input and output layer with one node each and two hidden layers with 50 nodes each. For all models the activation function for the output was the identity function, while the activation functions for the hidden layer were either sigmoid or ReLU. All models were trained using the Adam optimizer with learning rate 0.001, first moment parameter 0.9, and second moment parameter 0.999. The comparison of mean squared error as a function of epochs can be seen in figure 3. The corresponding fits can be seen in figure

Our implementation performed satisfactory. As expected, the different implementations did not perform exactly the same as the scikit-learn implementations, since scikit-learn uses additional optimizations.

Interestingly, our NN with sigmoid activations performed better than the scikit-learn implementation. We hypothesize that this is likely due to the difference in weight initialization. In both the ReLU and the sigmoid case, the scikit-learn models start of with lower mean squared errors. However, the scikit-learn MLP with sigmoid activation appears to get stuck in a local minima, which corresponds to a straight line fit (see figure 4).

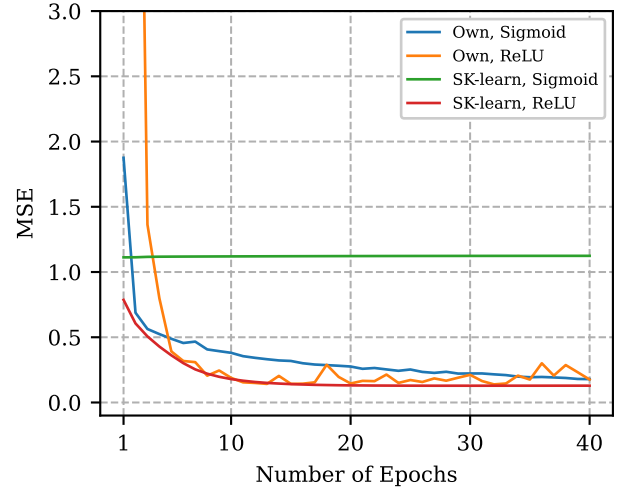


Figure 3: A comparison of the MSE as a function of the number of epochs for our implementation and scikit-learn. All models consist of two hidden layers with 50 nodes each. For gradient descent Adam with learning rate 0.001 and batch size 20 was used.

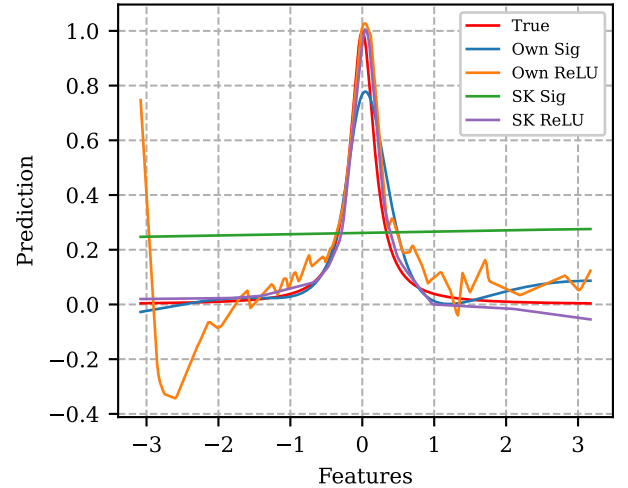


Figure 4: The fits produced by our and scikit-learn’s implementations of two 50-node hidden layers with sigmoid or ReLU activation functions after 40 epochs with Adam (learning rate 0.001, batch size 20).

The difference in initialization is particularly evident in the case of the ReLU models where the scikit-learn model achieves an MSE below 1 after just 1 epoch. In contrast, our ReLU model has an MSE of just over 400 after 1 epoch, but quickly approaches the performance of scikit-learn.

We also observe that our ReLU model shows greater variation in the MSE. While our ReLU model’s MSE is more unstable, the scikit-learn model is smooth. We also see that the fit is more erratic and has characteristic

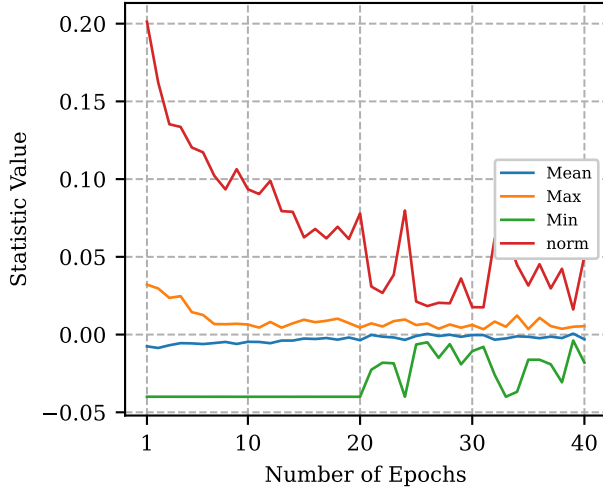


Figure 5: Various statistics of the change in weights of the bottom layer of our ReLU model as a function of epochs. Two hidden layers with 50 nodes, learning rate 0.001.

spikes. Although this could potentially be due to the poor initialization, we looked at the changes in weight between each epoch to see if there were signs of exploding gradients.

In figure 5, we see that while the gradients are unstable, we do not see unreasonable numbers. The change in bias behaved similarly. The sigmoid case also exhibited spikes although they were less pronounced. The behavior of the change in weight and bias might indicate that this behavior is not due to exploding gradients. However, we did not test implementing methods such as gradient-clipping to mitigate any possibly exploding gradients.

III. RESULTS AND DISCUSSION

A. Regression

For our regression analysis we configured two FFNNs. The first consisted of two hidden layers with 50 nodes each using sigmoid activation functions. For the output layer a simple linear activation function were used. The second model had a similar architecture, but with 100 nodes per hidden layer instead. We trained the two models with four different batch sizes and four different epochs, using ADAM as our optimizer for the gradient decent, and a conventional learning rate of 0.001. Finally predicting on the test set. The resulting MSE values can be seen in Figure 6 and Figure 7 respectively.

We see that both models perform similarly at their best, but for different batch sizes and epochs. The model with 100 nodes per hidden layer achieves its best result with a larger batch size (100) and for fewer epochs (200). This would generally mean faster computation time, but since the model have twice the number of nodes, it take

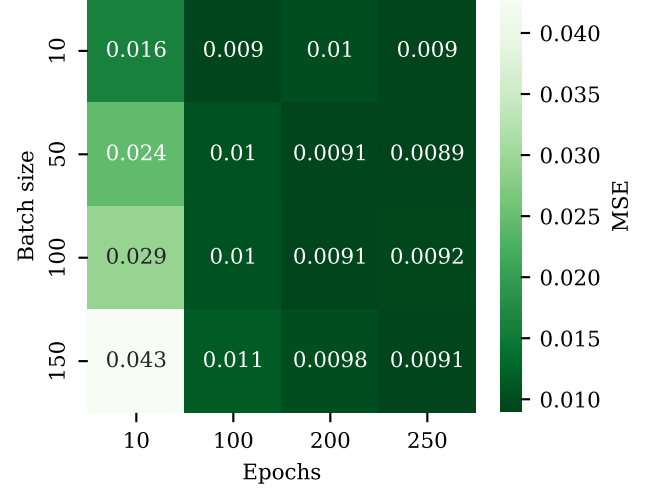


Figure 6: Regression performance of our FFNN model on the test set. The model consisted of two hidden layers with 50 nodes each using sigmoid activation functions.

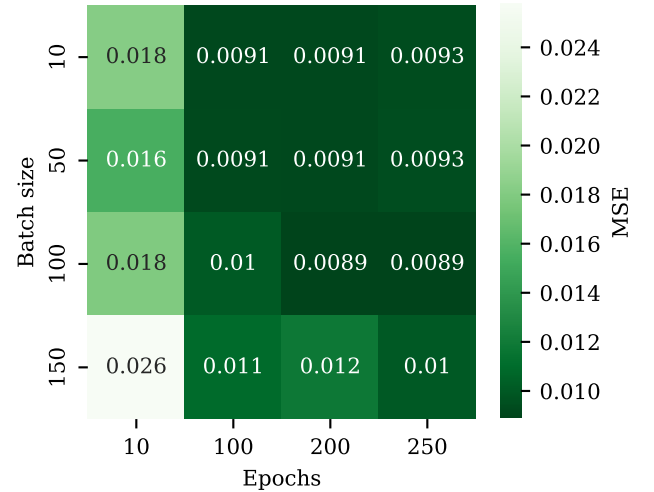


Figure 7: Regression performance of our FFNN model on the test set. The model consisted of two hidden layers with 100 nodes each using sigmoid activation functions.

twice as long to run on our system (~ 90 sec and ~ 30 sec respectively).

To see if there were any substantial performance benefits to be had with an increase in hidden layers, or with more nodes per layer, we configured a set of FFNNs with varying number of hidden layers and nodes. An architecture with one hidden layer were employed with 200 nodes, while the rest of the FFNNs consisted of 100 nodes each. We also used three different activation functions for the hidden layers, namely *ReLU*, *leaky-ReLU*, and *Sigmoid*. Then we trained the models and predicted on the test set. The resulting MSE values are plotted in Figure 8.

We see clearly that for our dataset the model consisting

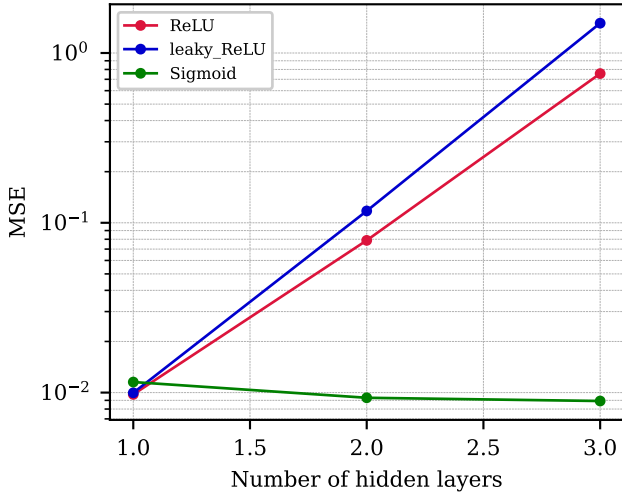


Figure 8: MSE on test prediction for different FFNN architectures. For the model with one hidden layer there were 200 nodes. The two other models with 100 nodes per layer. All models were trained with a batch size of 50, and for 250 epochs.

of two hidden layers with sigmoid as activation functions is the best choice. There are though a small decrees in the MSE for three hidden layers, but that would come with a computational time penalty.

As FFNNs really heavily on the gradient decent method during training we utilized two well known optimizers, namely *RMSProp* and *Adam*. Then compared them to the simplest method. We also considered different learning rates with the optimizers. The FFNN model consisted of two hidden layers with 50 nodes each, and the sigmoid as activation functions. The model were trained using a batch size of 150, running for 250 epochs. A graph of MSE values from predicating on the test set are shown in Figure 9. A more detailed heatmap of detailed values is to be found in Figure 17 in the Appendix.

While RMSProp show a more stable performance, Adam and the simpler model perform well for higher learning rates. The optimizer that gives the lowest MSE on test data in our case is the RMSProp with a learning rate of 0.001.

In conclusion we applied the best parameters and trained our model. Comparing it to one of the FFNNs we started out (two hidden layers with 100 nodes each) as well as the standard OLS fit. Our optimized FFNN architecture consisted of of two hidden layers with 50 nodes each, trained with a batch size of 150 and for 250 epochs. The predictions on test data are plotted in Figure 10.

We see that all three model predictions are able to capture the shape of the function closely at the center, but that both FFNNs are stable at the tails while the OLS model have a more erratic behavior. The optimized FFNN has an MSE on test data of 0.00861, while the OLS and the FFNN we start out have with 0.00946 and

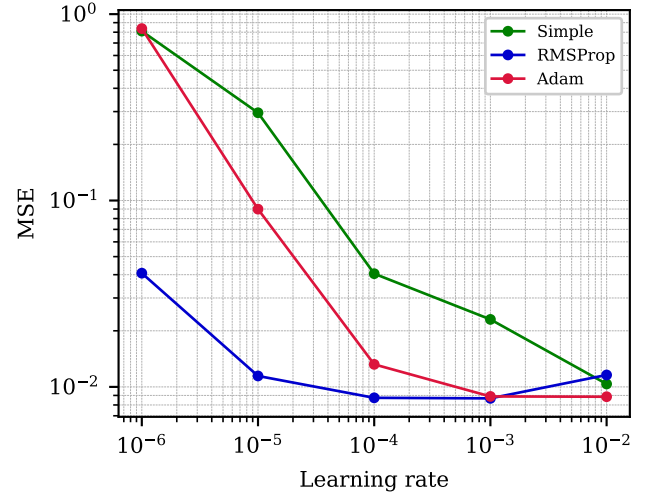


Figure 9: Model trained with different gradient optimizers. Two hidden layers, 50 nodes each. Batch size 50, and for 250 epochs.

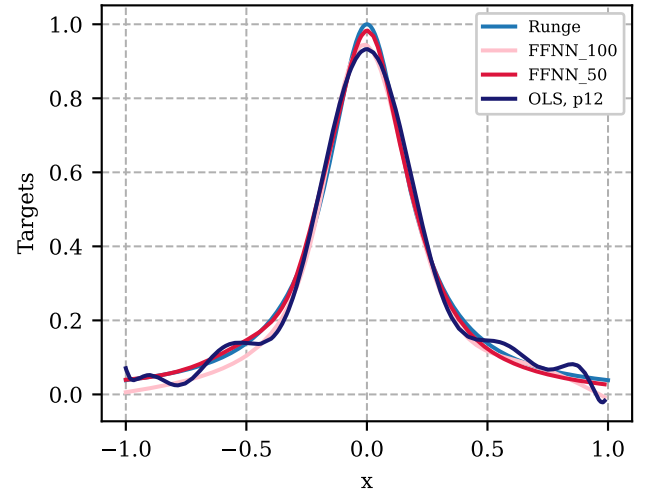


Figure 10: Model predictions on test set. Optimized model FFNN 50 and initial architecture FFNN 1000. The OLS model were trained on a feature matrix of degree 12.

0.00945 respectively.

B. Classification

For classification we used the full MNIST data set. The data was randomly split into training and test data. With test data proportion being 0.2. The input layers had 784 features, one for each pixel in the images. The output layer had 10 nodes, one for each class. The output activation function was softmax, and the cost function used was cross-entropy.

We first compared models with 1 and 2 hidden lay-

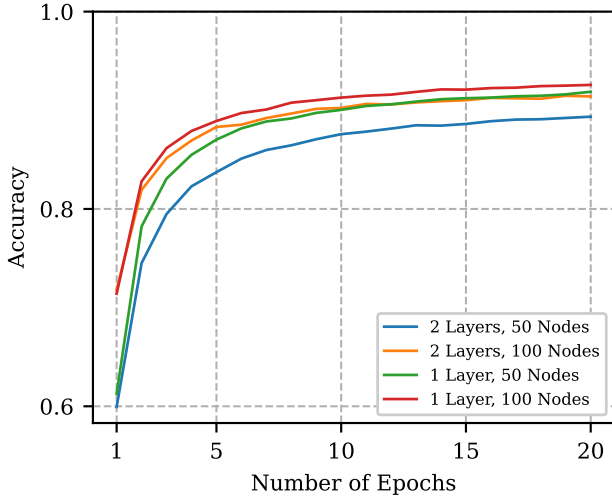


Figure 11: A comparison of accuracy as a function of epochs. Adam optimizer with learning rate 0.001 was used.

ers with 50 and 100 nodes by computing the accuracy as a function of the number of epochs. This was done for ReLU, sigmoid and leaky ReLU activation functions. In each case, training was done with batch size 200 and the Adam optimizer with learning rate 0.001, first moment decay 0.9 and second moment decay 0.999. No regularization was used. The resulting figure for the case of ReLU can be seen in figure 11.

We will denote the model with a layers with b nodes each as $aLbN$. Figure 11 shows that in the ReLU case 1L100N performs best and 2L50N performs worst. This is surprising as the least complex model, 1L50N, performs better than 2L50N, which should be more complex. This might be due to overfitting due to the excessive complexity of the higher model. However, this is undermined by the fact that 2L100N, which should be significantly more complex outperforms 1L50N initially and performs about equally good as the number of epochs increases. This, indicates that increasing the numbers of layers and the number of nodes have different effects beyond simply increasing the model complexity. The results were similar for the case of the sigmoid function. The corresponding figure can be seen in figure 18 in appendix C.

In the case of leaky ReLU, see figure 12, we get a different picture. Here the results follow our naive expectations and the most complex model performs the best. This indicates that the results shown in figure 11 might be due to vanishing gradients.

We also did a grid search with ReLU where we varied the number of layers and nodes. The results can be seen in figure 13. Here we see a clear trend that increasing the number of layers decreases the accuracy. For all node numbers, the model with only one layer performs best and increasing the number of layers lowers the accuracy uniformly. However, it is unlikely this is due to overfitting as the model with 1 layer and 50 nodes is outperformed

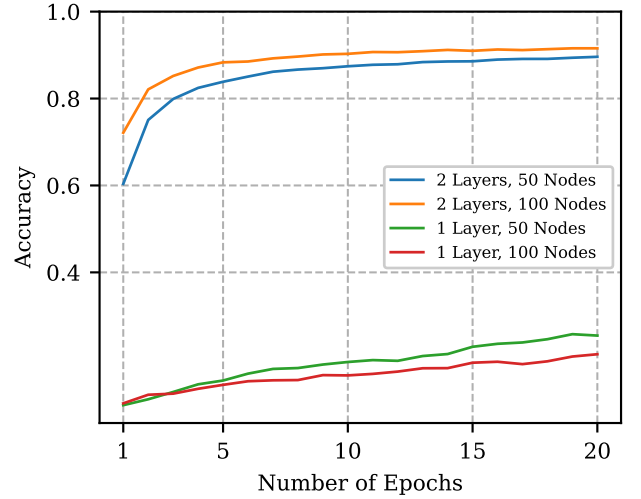


Figure 12: A comparison of accuracy as a function of epochs for different architectures using leaky ReLU activation functions. Adam optimizer with learning rate 0.001 was used.

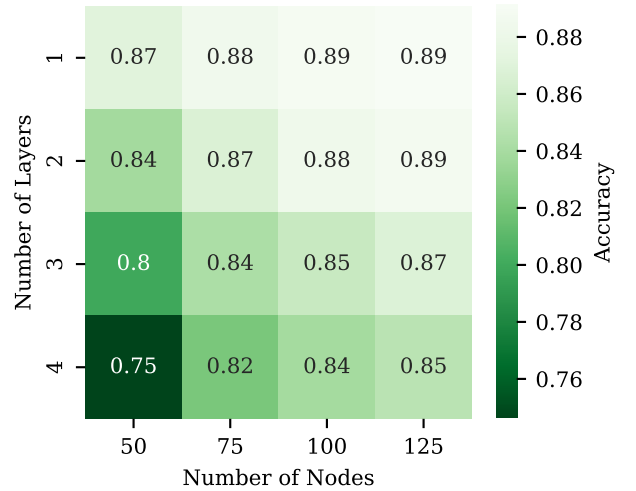


Figure 13: Caption

by or equal to more complex models, e.g. 3L125N. Hence, this indicates again that the issue might be vanishing gradients.

IV. CONCLUSION

In the regression case, we are able to model the underlying Runge function to a higher accuracy with a FFNN than with a standard OLS fit. However, considering the total computation time, in addition to tuning of parameters and hyperparameters, the OLS is not far off to justify this in our case.

In comparison, with scikit-learn we observed that ini-

tialization of weights and biases likely have a profound effect on the performance of FFNN models, especially for short training cycles.

We also found that ReLU and Sigmoid exhibit poor performance when increasing the number of layers, indicating vanishing gradients. This behavior was not ob-

served with leaky ReLU.

For future work we suggest looking at the effects of different methods of randomly initializing the weights and biases. In addition, analyzing the behavior of different activation functions, especially with respect to vanishing and exploding gradients and their effects.

-
- [1] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, et al., *Nature* **596**, 583–589 (2021).
 - [2] Y. Wang, M. Bennani, J. Martens, S. Racanière, S. Blackwell, A. Matthews, S. Nikolov, G. Cao-Labora, D. S. Park, M. Arjovsky, et al., *Discovery of unstable singularities* (2025), 2509.14185, URL <https://arxiv.org/abs/2509.14185>.
 - [3] J. M. Stokes, K. Yang, K. Swanson, W. Jin, A. Cubillos-Ruiz, N. M. Donghia, C. R. MacNair, S. French, L. A. Carfrae, Z. Bloom-Ackermann, et al., *Cell* **180**, 688–702 (2020).
 - [4] M. Hjorth-Jensen, *Applied Data Analysis and Machine Learning* (Department of Physics, 2025), URL https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/intro.html.
 - [5] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al., *Nature* **585**, 357 (2020), URL <https://doi.org/10.1038/s41586-020-2649-2>.
 - [6] D. Maclaurin, D. Duvenaud, M. Johnson, and J. Townsend., *Hips/autograd*, <https://github.com/HIPS/autograd> (2025).
 - [7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., *Journal of Machine Learning Research* **12**, 2825 (2011), URL <https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>.

Appendix A: Change in Weights and Biases

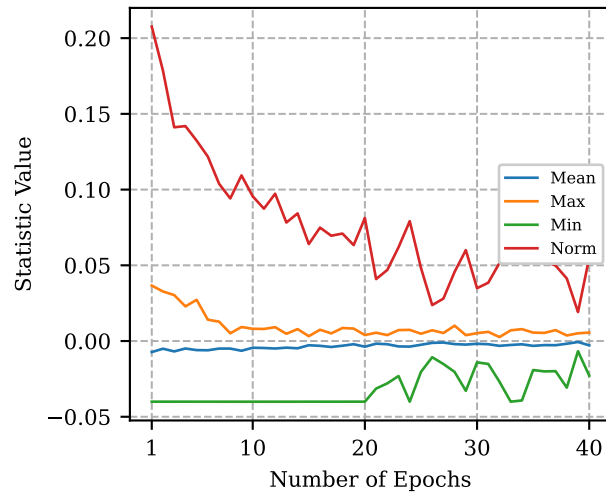


Figure 14: Various statistics of the change in biases of the bottom layer of our ReLU model as a function of epochs. Two hidden layers with 50 nodes, learning rate 0.001.

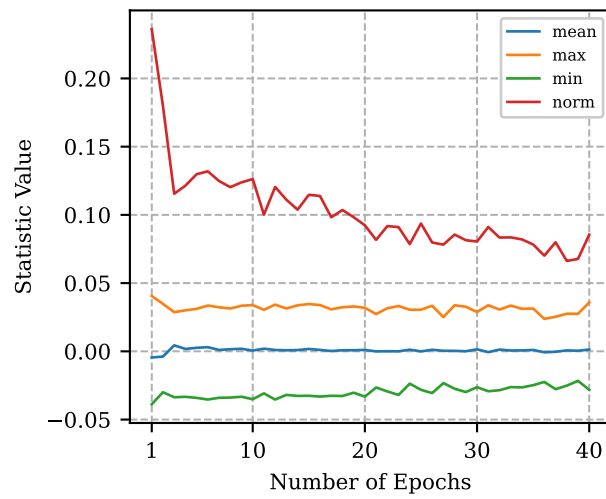


Figure 15: Various statistics of the change in weights of the bottom layer of our sigmoid model as a function of epochs. Two hidden layers with 50 nodes, learning rate 0.001.

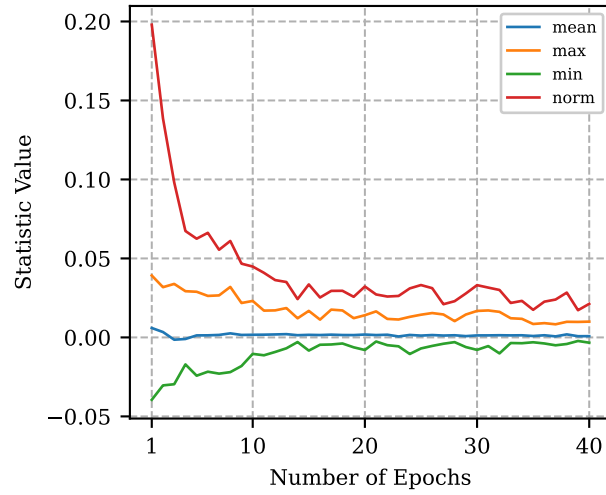


Figure 16: Various statistics of the change in biases of the bottom layer of our sigmoid model as a function of epochs. Two hidden layers with 50 nodes, learning rate 0.001.

Appendix B: Learning rates and optimizers

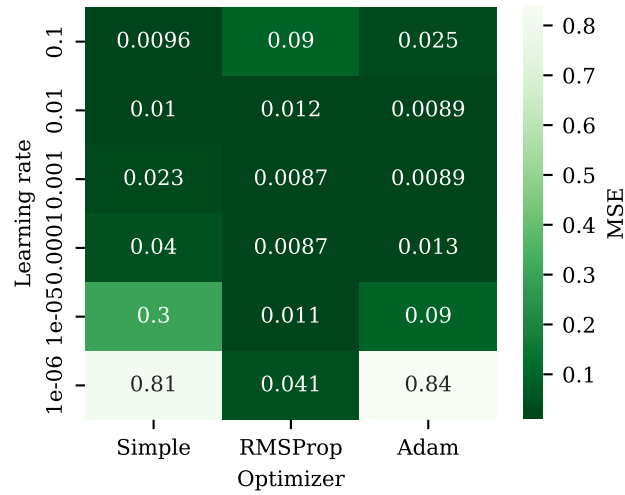


Figure 17: MSE on test prediction for various optimizers and learning rates. FFNN model consisting of 2 hidden layers with 50 nodes each using sigmoid activation functions. Model trained with a batch size of 50 and for 250 epochs.

Appendix C: Sigmoid Classifier Accuracy

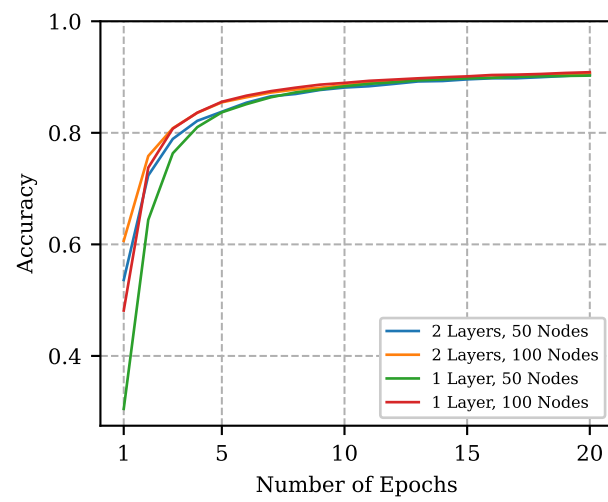


Figure 18: A comparison of accuracy as a function of epochs for different architectures using sigmoid activation functions. Adam optimizer with learning rate 0.001 was used.